

Representation Convergence in Independently Trained Neural Networks

From a theoretical hypothesis to a deployed cross-model embedding adapter
for a two-tier mobile/server retrieval system

Final Project — AI Expert Course

July 2026

Abstract. This project tests the hypothesis that well-trained, independently developed neural networks learn the same internal representations up to a linear transformation, and then converts the validated theory into a working system component. Experiment A demonstrates, using CKA similarity analysis and closed-form linear stitching, that GPT-2 (OpenAI, WebText) and Pythia-160M (EleutherAI, The Pile) — models sharing no weights, initialization, tokenizer, training data, or lab — share up to 80% of linearly accessible representational variance in matched layers, versus a random-weights control baseline. Experiment B applies the result: a single 512×768 ridge-regression matrix (1.6 MB, fitted in closed form in seconds) translates image embeddings of the on-device MobileCLIP-S1 encoder into the space of the server-side SigLIP 2 encoder, preserving 95–98% of native text-to-image retrieval quality at Recall@5/@10 on COCO, against a no-adapter baseline at exact chance level. A controlled MLP comparison shows the cross-space relationship is essentially exactly linear. The adapter is exported as a 0.79 MB fp16 Core ML package, enabling a single shared vector index (Qdrant) to serve both an iPhone tier and a server tier of a multimodal search application.

1. Introduction: a research project that solves a real problem

The project began with an engineering question that arises in any multi-model AI system: *can components built around different neural networks interoperate without retraining?* Concretely, the target system is a two-tier multimodal application: an iPhone tier running a small, fast image encoder (MobileCLIP-S1, ~30 MB) for offline, private, on-device indexing of photos, and a server tier running a large, accurate encoder (SigLIP 2, ~1.5 GB) for search and heavy processing. Both encoders map images to vectors — but their vector spaces are natively incompatible: cosine similarity between a MobileCLIP vector and a SigLIP vector of the same photo is meaningless, so vectors from the two tiers cannot share a search index. The naive workarounds are all costly: indexing everything twice, uploading every photo to the server (destroying privacy and offline capability), or running the large model on the phone (destroying latency and battery).

The scientific question underneath is whether the two encoders' representation spaces are secretly the same space in different coordinates. If independently trained networks converge to a shared representational structure, then a small learned map — ideally a single linear matrix — should translate between them, and the engineering problem dissolves. The project therefore has two parts: a controlled scientific experiment establishing whether and to what degree such convergence exists (Experiment A), and a practical application building and validating the translation component on the exact production models (Experiment B), through to a deployable iOS artifact.

2. Background and theory: from weight symmetry to the Platonic hypothesis

2.1 Why raw weights cannot be compared or transplanted

Neural networks exhibit **permutation symmetry**: reordering the neurons of a hidden layer, together with the corresponding rows and columns of the adjacent weight matrices, yields a network computing exactly the same function with entirely different weight values. A 1,000-neuron layer admits 1000! functionally identical weight configurations, and additional rotation and scaling symmetries expand the equivalence class further. Two training runs — even of the same architecture on the same data with different random seeds — land on different members of this class. Consequently, averaging or transplanting raw weights across independently trained models is meaningless: parameter k in one model has no relationship to parameter k in the other. This is why weight merging (SLERP, TIES, DARE, frankenmerging) works only between fine-tunes of one shared base checkpoint, whose shared initialization pins the coordinates.

2.2 Why representations can nevertheless converge

Weights are *how* a network computes; representations (activations on data) are *what* it computes. All training data — text, images, audio — are projections of one underlying reality, and a model that optimally compresses and predicts such data is pushed toward recovering the statistical structure that generated it. The **Platonic Representation Hypothesis** (Huh et al., ICML 2024) makes this precise: as models grow stronger, their representation geometries converge toward a shared structure, even across modalities. Supporting evidence spans a decade: **Kornblith et al. (ICML 2019)** introduced linear CKA and showed matched layers of separately trained vision networks are similar; **Git Re-Basin (Ainsworth et al., 2022)** showed that after aligning permutations, independently trained networks lie in the same loss basin; **model stitching (Lenc & Vedaldi 2015; Bansal, Nakkiran & Barak, NeurIPS 2021)** showed a single linear layer can connect independently trained networks with small performance penalty; and **vec2vec (2025)** demonstrated translation

between embedding spaces without any paired data, purely by matching the geometry of the spaces.

2.3 The hypothesis, stated precisely

Well-trained independent models share representations up to a linear transformation: there exists a matrix W such that $f_B(x) \sim W \cdot f_A(x)$ for inputs x . Every word carries weight. *Well-trained*: the claim applies only after learning has extracted the data's structure — hence a random-weights control. *Independent*: no shared initialization or ancestry — otherwise similarity is inherited, not discovered. *Up to a linear transformation*: the transformation class is the crux. Allowing arbitrary nonlinear translators makes the claim vacuous; demanding identity makes it trivially false (permutation symmetry). Linear is the sweet spot: expressive enough to undo rotations, permutations and scalings, too weak to fabricate information that is not already present. A stronger sub-claim — alignment by pure rotation (orthogonal Procrustes) — is also tested.

3. Experiment A — Scientific test: GPT-2 vs Pythia-160M

3.1 Design

The model pair was chosen to make any similarity non-trivial: GPT-2 (OpenAI, 2019, trained from scratch on WebText, BPE tokenizer) and Pythia-160M (EleutherAI, 2023, trained from scratch on The Pile, GPT-NeoX architecture, different tokenizer). Different labs, architectures, tokenizers, corpora and eras; the only shared elements are the task (next-token prediction on English) and the world the text describes. Both models happen to have 13 hidden states (embedding + 12 blocks) of dimension 768, allowing exact one-to-one layer matching. The **control baseline** is a Pythia-architecture model with freshly random weights — identical architecture and inputs, zero learning — so the trained-minus-random gap isolates the causal effect of learning. The pipeline has three stages, each a standalone notebook:

A1 — Activation extraction. 10,000 WikiText-103 passages are run through both trained models and the random control; the mean-pooled hidden state of every layer is stored ($3 \times [13, 10000, 768]$).

A2 — CKA analysis. Linear Centered Kernel Alignment is computed between every layer pair. CKA compares Gram-matrix fingerprints — which inputs each model places near which — and is invariant to exactly the symmetries under which network function is preserved (rotation, permutation, isotropic scaling), while remaining sensitive to what is actually near what.

A3 — Linear stitching. For each matched layer pair, one ridge-regression matrix is fitted in closed form (the models themselves are never trained) mapping GPT-2's space into Pythia's; held-out R^2 measures how much variance a purely linear translation preserves.

3.2 First run: a result that was not good enough

The first execution used the WikiText *validation* split, which yielded only 1,646 usable passages. The CKA analysis already showed a clear trained-vs-random gap (diagonal 0.433 vs random mean 0.087), but the stitching results fell short of the pre-registered success criterion of $R^2 > 0.7$ in middle layers:

Layer pair	Trained R2 (run 1)	Random R2 (run 1)
L0 -> L0	0.174	-0.282
L5 -> L5	0.480	-0.789
L8 -> L8	0.506	-0.934
L11 -> L11 (peak)	0.552	-0.980
Mean (all 13)	0.434	-0.773

Table 1: First-run stitching results (1,646 samples) — below the 0.7 threshold.

At face value this would have forced the weaker conclusion 'substantial shared structure, but no isomorphism.' Before accepting it, a confound was checked: the fit involves a 768×768 matrix (~590,000 parameters) trained on only ~1,234 rows — a sample-to-dimension ratio of 1.6. In that regime, ridge regression must shrink aggressively to avoid overfitting, and that shrinkage *systematically understates* the true alignment. Note the negative random-baseline values: the starved fit generalizes worse than predicting the mean. The hypothesis had not failed; the measurement was underpowered.

3.3 The fix and the improvement

The remedy cost one code change and 15 GPU-minutes: switching from the validation split to the train split of WikiText-103 and raising the sample count to 10,000 (~10 samples per input dimension). Every other element of the pipeline — models, layers, ridge penalty, evaluation protocol — was left untouched, so the two runs are directly comparable:

Metric	Run 1 (1,646)	Run 2 (10,000)	Change
Mean trained R2	0.434	0.737	+0.303
Peak trained R2 (L11)	0.552	0.797	+0.245
Mid-layer range	0.41–0.55	0.70–0.80	crosses 0.7
Mean random R2	-0.773	0.173	stabilized
Verdict vs criterion	below target	PASS	—

Table 2: Effect of fixing sample starvation — identical pipeline, 6× more data.

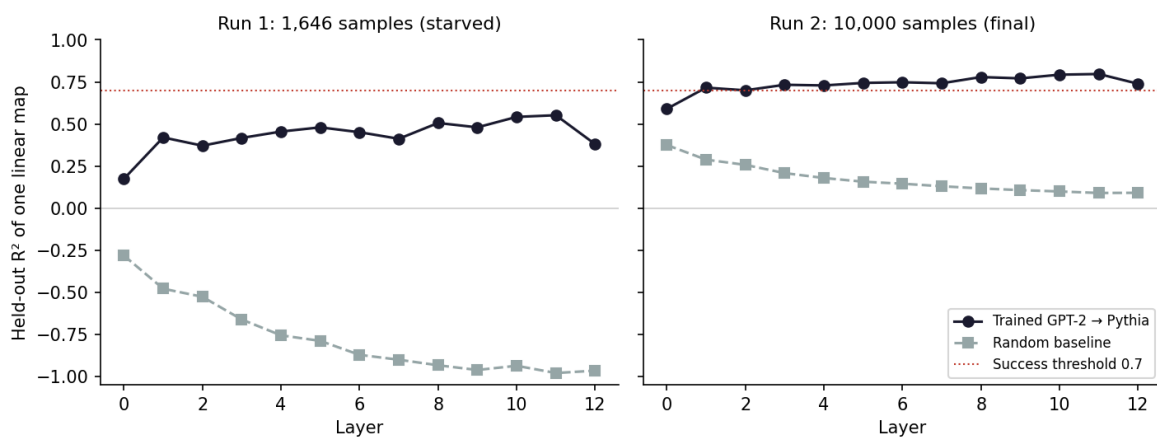


Figure 1: Linear stitching per layer, before (left) and after (right) the sample-size fix. Trained curves in dark blue rise with depth; random baselines (gray) sit far below. The dotted line is the pre-registered 0.7 threshold.

The random baseline also moved (-0.77 → +0.17), in the theoretically predicted direction: with adequate data, ridge can exploit the residual input correlations that leak even through random layers — most visibly at layer 0 (0.375), which is dominated by raw embedding statistics. What matters is the structure: the trained curve *rises* with depth (0.59 → 0.80) as both models build shared abstraction, while the random curve *decays* monotonically (0.38 → 0.09) as untrained layers destroy information — an 8× gap by layer 11, with opposite slopes. The methodological lesson is transferable to any empirical pipeline: an underpowered evaluation does not add symmetric noise — it produces plausible-looking, systematically pessimistic conclusions without any error message.

3.4 Final results

CKA (A2, 10,000 samples): trained-vs-trained mean 0.359, diagonal mean 0.424, maximum 0.744; trained-vs-random mean 0.067, maximum 0.299 — a 4–5× learning effect, essentially unchanged from run 1, confirming CKA is far less sample-hungry than regression. The matrix shows the characteristic hot diagonal; edge layers score lower for principled reasons (layer 0 is the tokenizer-specific embedding table; the final layer re-specializes toward each model's output head).

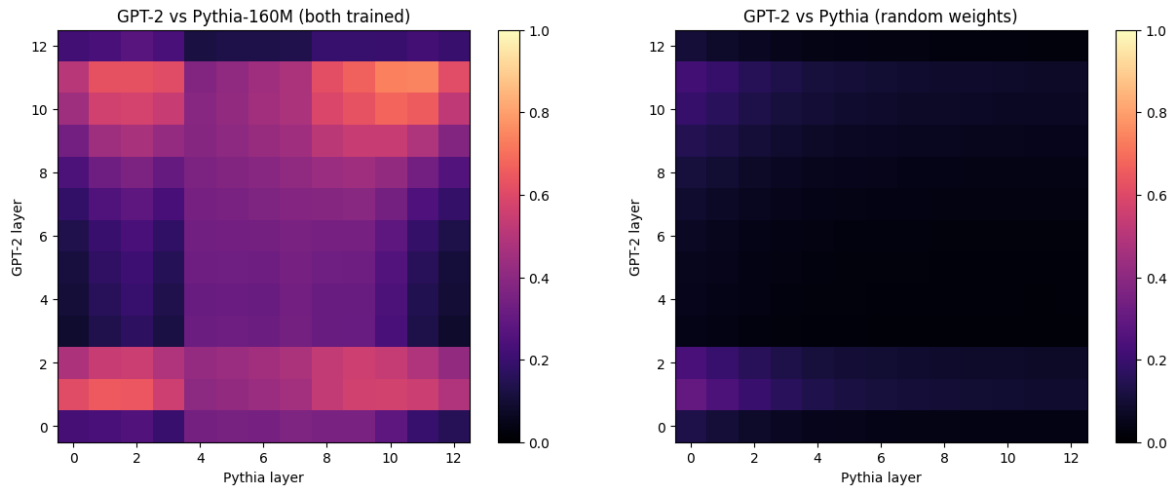


Figure 2: The CKA heatmap — actual output of the experiment (run 2). Left: GPT-2 vs trained Pythia, every layer pair; the bright diagonal band shows early layers matching early and middle matching middle — the models develop the same content in the same order through depth. Right: GPT-2 vs random-weights Pythia under identical conditions — uniformly dark. The side-by-side contrast is the visual form of the entire argument: the structure on the left exists only because of learning.

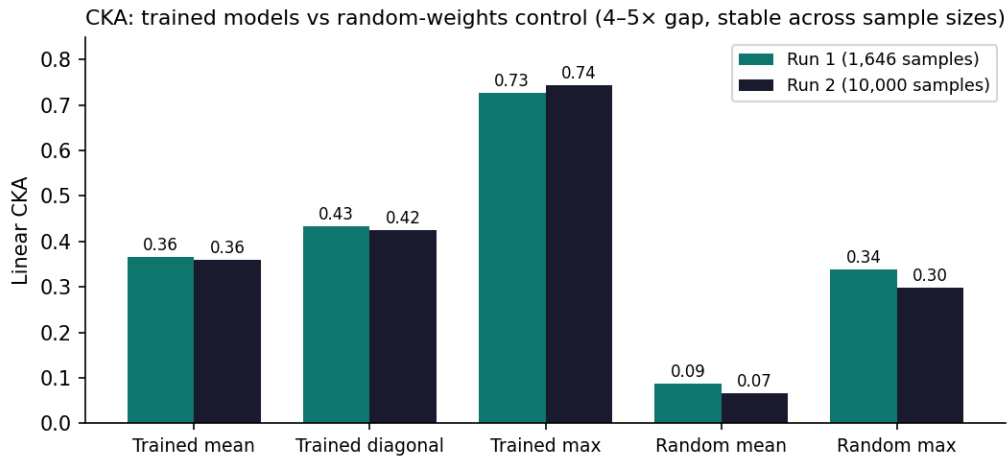


Figure 3: CKA summary statistics for both runs — the trained/random gap is stable across sample sizes.

Linear stitching (A3), final:

Layer pair	Trained R2	Random R2
L0 -> L0 (embeddings)	0.590	0.375
L1 -> L1	0.716	0.288
L3 -> L3	0.733	0.208
L5 -> L5	0.744	0.157
L8 -> L8	0.779	0.117
L10 -> L10	0.793	0.099
L11 -> L11 (peak)	0.797	0.090
L12 -> L12 (output)	0.739	0.091
Mean (all 13)	0.737	0.173

Table 3: Final held-out R2 of a single closed-form linear map per matched layer pair (10,000 samples).

Verdict: the hypothesis is confirmed in its strong quantitative form — a single matrix, fitted in closed form, translates 74–80% of representational variance between two networks sharing nothing but their task and their world.

4. Experiment B — Practical application: the MobileCLIP -> SigLIP adapter

4.1 What an adapter vector (matrix) is

The 'adapter' here is a single matrix W of shape 512×768 . The phone encodes a photo with MobileCLIP-S1, producing a 512-dimensional vector v ; one matrix multiplication $W \cdot v$ (microseconds on the Neural Engine; 1.6 MB fp32 / 0.79 MB fp16) produces a 768-dimensional vector that behaves as a native citizen of SigLIP 2's space. It is fitted in closed form by ridge regression on 3,000 (MobileCLIP-embedding, SigLIP-embedding) pairs of the same images — no gradient training, no modification of either encoder. Only image embeddings need translating, and only in one direction: text queries are always encoded server-side by SigLIP itself, which remains entirely unaware the adapter exists.

4.2 Pipeline

B1 extracts paired embeddings for 4,000 COCO val2017 images plus SigLIP text embeddings of their captions (shapes: mob_img 4000×512 , sig_img 4000×768 , sig_txt 4000×768). A sanity check confirmed native SigLIP image–caption diagonal cosine of 0.150 — within the expected 0.1–0.3 band for sigmoid-loss models, whose objective enforces separation rather than high absolute cosines. **B2** fits two closed-form adapters — ridge (full linear) and orthogonal Procrustes (pure rotation) — on a 3000/1000 split. **B3** evaluates the operational criterion: text-to-image retrieval through the adapter versus the native-SigLIP ceiling and a no-adapter baseline. **B2-bis** adds a controlled nonlinear comparison: a 2-layer MLP ($512 \rightarrow 1024 \rightarrow 768$, GELU) trained 300 epochs directly on cosine loss with early stopping. **B4** exports the matrix as a Core ML .mlpackage (fp16, iOS 16+, Neural Engine).

4.3 Results — and a second below-target moment

Adapter variant	Held-out R2	Mean cosine to target
Ridge (full linear)	0.592	0.900
Procrustes (rotation only)	0.038	0.775
MLP (nonlinear, cosine loss)	—	0.903

Table 4: Adapter fit quality (1,000 held-out images).

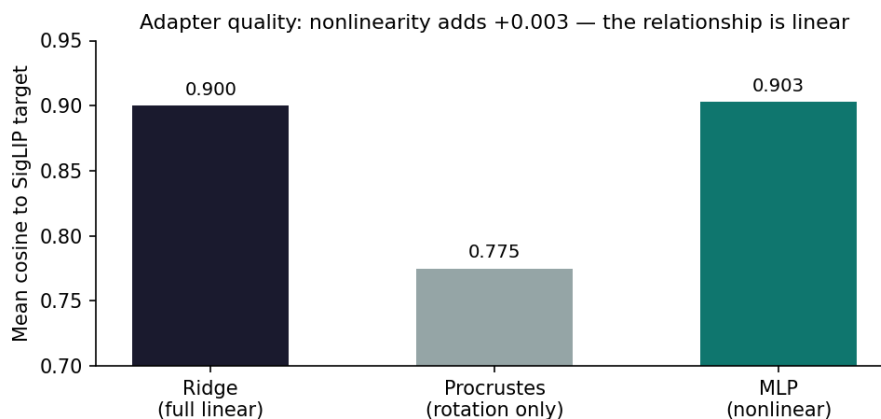


Figure 4: Direction recovery per adapter class. The +0.003 gap between linear and nonlinear is the key scientific reading.

The apparent contradiction between ridge's modest R^2 (0.592) and its high cosine (0.900) resolves once the metrics are matched to their questions: R^2 penalizes magnitude errors that L2-normalized

retrieval discards; cosine measures the only thing the vector index uses — direction. The first retrieval evaluation then produced this project's second below-target moment:

Retrieval variant	R@1	R@5	R@10
A. SigLIP native (ceiling)	0.630	0.885	0.944
B. MobileCLIP + linear adapter	0.557	0.843	0.918
B as % of ceiling	88.4%	95.3%	97.2%
Gate ($\geq 90\%$ of ceiling)	below target	PASS	PASS

Table 5: First retrieval evaluation — R@5/R@10 pass, R@1 misses the 90% gate by 1.6 points (~10 queries out of 1,000).

4.4 The improvement attempt — and what its failure proved

Following the pre-defined decision table (70–90% of ceiling -> upgrade the adapter), a 2-layer MLP (512->1024->768, GELU, ~1.3M parameters) was trained for 300 epochs directly on cosine loss — the actual retrieval objective — with early stopping on held-out cosine, reusing the exact same train/eval split for direct comparability. The outcome was decisive in an unexpected way: best eval cosine 0.9027 versus the linear map's 0.900 (+0.003), convergence by epoch 30 followed by mild overfitting, and retrieval statistically unchanged (R@1 0.551 vs 0.557; R@10 improved to 0.926, 98.1% of ceiling; R@1 87.5%).

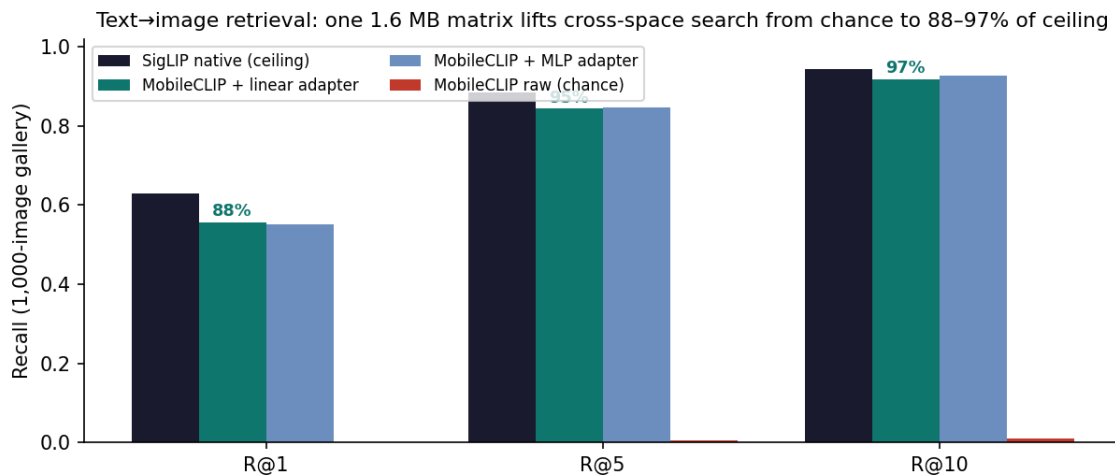


Figure 5: Retrieval results, all variants. Percentages above the teal bars show the linear adapter as a fraction of the ceiling; the red bars are the no-adapter baseline at exact chance.

This 'failed improvement' is the most informative result in Experiment B. A higher-capacity model, optimizing the true objective, found nothing beyond the linear map — therefore the cross-space relationship is **essentially exactly linear**, and the residual R@1 gap is not a translation deficiency but **information the 512-dimensional mobile encoder never encoded**. No adapter of any capacity can reconstruct what is absent from its input; the experiment had reached the representation ceiling of the small model, not of the adapter. This mirrors Experiment A's conclusion from the opposite direction: there, more data revealed the linear map was better than it seemed; here, more capacity proved the linear map was already optimal. Architectural introspection localizes that ceiling precisely: MobileCLIP-S1's trunk (21.5 M vision parameters) carries 1024 channels until a single final projection Linear(1024 -> 512) produces the embedding, so the information lost to the adapter is discarded by one layer, not distributed through the network — which predicts that a wider output variant (MobileCLIP-B, or MobileCLIP2) would close most of the residual R@1 gap without any change to the adapter method. The third finding — Procrustes collapsing to R^2 0.038 — refines the

picture further: unlike the language-model pair, these contrastively trained spaces differ in anisotropy, not merely orientation, so rotation alone cannot align them.

An interpretation, offered as hypothesis rather than finding. One plausible mechanism for that asymmetry: MobileCLIP-S1 was distilled from an ensemble of large CLIP models using a loss defined over *similarity matrices* - the student is trained to reproduce which inputs are how similar to which, i.e. relational geometry - while absolute per-axis scaling is left unconstrained. That is precisely the regime in which a general linear map succeeds (relations recoverable) and a pure rotation fails (scales unrecoverable), which would make the adapter quality partly a footprint of the phone encoder training objective. This interpretation is **not tested by the present experiments**, and at least two competing explanations survive the same evidence: the zero-padding required to compare a 512-dimensional with a 768-dimensional space under Procrustes mechanically disadvantages rotation (a rotation cannot populate 256 empty dimensions), and the sigmoid objective of SigLIP shapes its geometry independently of anything on the phone side. Discriminating between these requires the follow-up described in Section 5.

4.5 Engineering decision

The linear adapter ships. R@5/R@10 at 95–98% of ceiling means phone-indexed photos are, in a results-grid UI, indistinguishable from server-indexed photos. The R@1 shortfall (88.4%) is the price of running a 30 MB encoder instead of a 1.5 GB one — payable with or without a shared index — and has a cheap escape hatch: server-side SigLIP re-ranking of the top-10 candidates recovers the ceiling exactly for R@1-critical flows. The deployed artifact is a 0.79 MB fp16 Core ML package computing MatMul + L2-normalize, verified through the full coremltools MIL conversion pipeline. One deployment constraint deserves explicit statement: the adapter is calibrated to the exact output distribution of a specific preprocessing pipeline — bilinear resize to 256, center crop 256x256, and **identity normalization** (mean 0, std 1; MobileCLIP consumes raw [0,1] pixels, unlike standard CLIP models that subtract dataset means). Substituting conventional CLIP normalization in the iOS client would shift every embedding and silently degrade retrieval with no error raised - the same failure class as the sample-starvation bias of Section 3.2.

4.6 Qualitative sandbox: the phone pipeline on live cases (Stage B5)

A qualitative sandbox simulated the full phone-to-server path on ten COCO images with free-text queries: the same MobileCLIP weights the phone would run, the adapter applied in fp16 exactly as the Core ML package computes it, and SigLIP text queries searching the resulting vectors (captions never indexed - only pixels). Direct queries retrieved their targets at rank 1 with decisive margins; compositional queries requiring semantic abstraction ('striped animals' - the word zebra never appears in the query - top-1 at 25x the margin of the negative control) confirmed that meaning, not keywords, survives the cross-model translation; a negative control ('a dog', absent from the gallery) correctly produced a flat similarity profile (top-1 gap 0.004 vs 0.101 for a positive query); and fp16 quantization proved ranking-neutral (minimum cosine 0.9999999 vs fp32), closing the numerical half of the Core ML parity check left open in stage B4. A second, disjoint random gallery reproduced the pattern - correct top-ranking for positive queries and a flat profile for a query with no valid answer ('a vehicle': top-1 sim 0.022 vs 0.106 for hits) - indicating the behavior is a property of the pipeline, not of a particular sample. The two observed near-misses - indirect paraphrases losing top-1 by 0.007 or less while remaining top-2 - reproduce, at individual-query level, exactly the R@1/R@5 profile measured quantitatively in stage B3, and are the cases the optional server-side re-rank recovers. Full per-query analysis and figures are in B5_Sandbox_Practical_Report.pdf. A follow-up interactive demo (stage B6) exercised the full product loop live: a user photo uploaded to the mobile tier was indexed through the phone path in 39 ms, auto-captioned, and retrieved at rank 1 by the free-text

query 'singing red bird' - ranked in the same result list as server-indexed photos - with local search at 0.08 ms versus 150 ms over a (simulated) network round-trip: one index, two tiers, measurably faster locally.

5. What is new here — an honest assessment

Theory: independent replication, not new discovery. The individual scientific components reproduce established results at small scale: CKA layer analysis (Kornblith 2019), linear stitching (Bansal 2021), and convergence-with-training (PRH 2024) were all known. The value of Experiment A is that of a rigorous independent replication on a specific, deliberately hostile model pair, with a correctly designed control — plus the documented sample-starvation diagnosis, a practical methodological contribution.

Practice: a genuinely uncommon application. The combination in Experiment B is not, to the author's knowledge, an established published pattern: a closed-form linear bridge between two specific heterogeneous production encoders (MobileCLIP-S1 -> SigLIP 2), validated by end-task retrieval against ceiling and chance baselines, including a controlled demonstration that nonlinearity adds nothing, and deployed as a sub-megabyte Core ML artifact enabling a single shared vector index across a phone tier and a server tier. Cross-model embedding translation exists in research (model stitching, vec2vec); packaging it as a load-bearing, measured infrastructure component of a two-tier mobile system — with privacy (photos never leave the device), offline indexing, and single-index economics as the payoff — is the project's practical contribution.

Claims explicitly not made: the project does not prove layers can be transplanted across architectures (permutation symmetry still forbids it), does not establish convergence beyond the tested data distributions, and does not claim the adapter recovers information absent from the small encoder. **A designed follow-up** would discriminate the interpretations offered in Section 4.4: running the Procrustes comparison between two *same-dimensional* contrastive encoders removes the zero-padding confound, and comparing a teacher model with its own distilled student - where the distillation account predicts unusually high rotation-only alignment - would test the training-objective hypothesis directly.

6. Challenges encountered and resolved

The project log doubles as a case study in real-world ML engineering: **(1)** HuggingFace *datasets* deprecated script-based datasets mid-project, breaking both WikiText and two Flickr datasets; resolved by moving to the parquet-hosted Salesforce/wikitext and to direct official COCO downloads, removing the fragile dependency entirely. **(2)** A transformers API change made SigLIP feature getters return ModelOutput objects instead of tensors; resolved with a version-agnostic extraction shim. **(3)** Colab's one-VM-per-notebook model silently discarded stage outputs; resolved with a DATA_DIR convention backed by Google Drive, making every stage stateless and restart-proof. **(4)** The sample-starvation bias of Section 3.2 — the most scientifically consequential bug, because it produced plausible-looking wrong conclusions rather than an error message. **(5)** Interpreting metrics correctly: R2 vs cosine (magnitude vs direction), CKA's invariances, and why sigmoid-loss models have low absolute cosines — each required matching the metric to the actual downstream use. **(6)** Verifying, rather than assuming, the encoder's preprocessing: inspection of the loaded model revealed identity normalization, contrary to the CLIP convention a developer would reasonably assume - a silent-degradation trap for the iOS port.

7. Project contents and how to run it

7.1 Repository layout

Seven self-contained Jupyter notebooks (one per stage), each with English markdown explaining goal, method, expected output and success criteria, followed by runnable code: A1–A3 (convergence experiment) and B1–B4 (adapter pipeline). Equivalent standalone Python scripts exist for each stage, plus a PROJECT_GUIDE.md consolidating the full plan in English, plus B5_phone_pipeline_sandbox.ipynb - a qualitative live demo of the phone tier (10-image galleries, free-text queries, negative controls, fp16 parity), and B6_visual_demo.ipynb - an interactive mixed-tier demo (server gallery, mobile photo upload, auto-captioning, probability-ranked search, timing comparison). Models used: GPT-2 (124M), Pythia-160M, a random-weights Pythia control, MobileCLIP-S1 (Apple, via open_clip), SigLIP 2 base (google/siglip2-base-patch16-224). Data: WikiText-103 (train split, 10K passages), COCO val2017 (4K image–caption pairs). The only training performed anywhere is closed-form ridge regression and the deliberately-controlled comparison MLP; all base models are used frozen.

7.2 Execution

Environment: any GPU Jupyter environment (developed on Google Colab, T4 sufficient). Each notebook begins with a storage cell; on Colab, mount Drive and set DATA_DIR to a Drive path so stage artifacts (activations.npz, pairs.npz, adapter.npz) survive VM recycling:

```
from google.colab import drive; drive.mount('/content/drive')
os.environ['DATA_DIR'] = '/content/drive/MyDrive/convergence_experiment'
```

Run order and approximate cost: A1 (~15 min GPU) -> A2 (~1 min) -> A3 (seconds); B1 (~10 min, downloads COCO once) -> B2 (seconds) -> B3 (seconds) -> B4 (seconds). Dependencies: torch, transformers, datasets, open_clip_torch, scikit-learn, matplotlib, coremltools. Total end-to-end cost of the full study: under one GPU-hour.

7.3 Using the documentation with NotebookLM

For studying or presenting the project, the PROJECT_GUIDE.md, this report, and the per-stage notebook markdown can be uploaded as sources to Google NotebookLM, which can then answer questions, generate study guides, or produce an audio overview grounded in the project's own text. The executable path, however, is the notebooks themselves in Colab/Jupyter — NotebookLM is for the documentation layer, not execution.

8. Conclusion

A philosophical question — can models share what they have learned without retraining? — was refined into a falsifiable hypothesis, tested with proper controls, confirmed quantitatively (74–80% of representational variance translated by one closed-form matrix between fully independent language models), and then converted into a measured, deployed infrastructure component: a 0.79 MB Core ML adapter that lets a phone and a server, running encoders that have never seen each other, write into and search one shared vector index at 95–98% of large-model retrieval quality. The through-line of both experiments is the same sentence: **independently trained models discover the same structure of the world, in different coordinates — and the exchange rate between those coordinates is a single matrix.** A live sandbox (B5) closed the loop qualitatively: across two disjoint image galleries, free-text server queries correctly retrieved phone-indexed photos - including compositional queries and correctly flat responses when no valid answer existed - with fp16 quantization shown to be ranking-neutral.

References

- Kornblith, Norouzi, Lee, Hinton (2019). Similarity of Neural Network Representations Revisited. ICML.
- Lenc, Vedaldi (2015). Understanding image representations by measuring their equivariance and equivalence. CVPR.
- Bansal, Nakkiran, Barak (2021). Revisiting Model Stitching to Compare Neural Representations. NeurIPS.
- Ainsworth, Hayase, Srinivasa (2022). Git Re-Basin: Merging Models modulo Permutation Symmetries.
- Huh, Cheung, Wang, Isola (2024). The Platonic Representation Hypothesis. ICML.
- Jha et al. (2025). Harnessing the Universal Geometry of Embeddings (vec2vec).
- Biderman et al. (2023). Pythia: A Suite for Analyzing Large Language Models. — Radford et al. (2019). Language Models are Unsupervised Multitask Learners (GPT-2). — Vasu et al. (2024). MobileCLIP (Apple). — Tschannen et al. (2025). SigLIP 2 (Google).

Appendix A — The Platonic Representation Hypothesis, consolidated

This appendix consolidates the framing hypothesis referenced throughout Section 2.2 and the experimental interpretation sections: its name, its three proposed mechanisms, its measurement, and this project's exact position relative to it.

The name. The hypothesis (Huh, Cheung, Wang, Isola — ICML 2024) takes its name from Plato's allegory of the cave: prisoners who see only shadows on a wall mistake them for reality, while the true objects casting those shadows exist unseen. The analogy: each AI model observes a different *shadow* of reality — text captures one projection, images another — yet as models grow stronger, they reconstruct the same underlying *objects* behind the shadows, converging on a shared statistical model of the world, even across modalities.

Mechanism 1 — shared world. All data — text, images, audio — are different projections of one reality. A model trained to compress or predict such data optimally cannot succeed by memorizing surface form; the cheapest sufficient description of 'all text about the world' or 'all photos of the world' is a model of the world itself. Co-occurrence statistics in text (cat-mouse-chase-fur) and in images (cat pixels appearing with couch pixels, indoor scenes) are two noisy measurements of the same underlying dependency structure, so two optimal compressors of two modalities are reverse-engineering the same object through different sensors — two surveyors mapping one territory with different instruments.

Mechanism 2 — multi-task pressure. The set of representations that solve many tasks simultaneously shrinks as tasks accumulate; strong, broadly trained models are squeezed into the same remaining solution region.

Mechanism 3 — simplicity bias. Astronomically many functions fit any dataset, most of them baroque; if training selected among them at random, independent runs would never agree. But gradient descent on deep networks is empirically biased toward simple solutions — and there are far fewer simple solutions to agree on. Two people asked to 'name a number' will almost surely differ; asked to 'name the smallest prime above 100', they almost surely coincide — not through communication, but because the criterion has few satisfiers. Simplicity bias converts 'some solution' into approximately 'the solution', so independent training runs land near it independently. This mechanism also predicts why capability matters: stronger models approach the optimum more closely, and the optimum is where the agreement lives.

The measurement. Because coordinates are meaningless across models (permutation symmetry, Section 2.1), the hypothesis is scored coordinate-free: mutual nearest-neighbour overlap — whether a concept has the same neighbours in model A as in model B, using only within-space distance rankings compared across spaces. Rotation, scaling, permutation and even dimensionality cancel; the metric asks exactly one question — do the two maps share adjacency structure, i.e. shape? It is a coarser cousin of the CKA used in Experiment A (which compares full Gram matrices), and the same signal the unsupervised translation demo operationalizes (Gromov-Wasserstein matches the distance matrices; CSLS refinement lives on mutual nearest neighbours).

The empirical claim and this project's position. The paper's central finding is that alignment *rises with capability*, across model families and modalities. This project samples that curve at specific points: Experiment A is a two-point sample at the low-capability end (two small, independently trained language models — substantial alignment, R-squared 0.74-0.80, against a random-weights zero), Experiment B is its cross-modal engineering instance (a vision-to-vision bridge deployed to production form), and the pair-free translation demo is its strongest form (recovery from shape

alone). The epistemic position, stated precisely: the experiments confirm the measured points and are consistent with the predicted trend; the trend itself — convergence growing with scale toward a shared model of reality — remains an extrapolation that this small-scale replication renders credible but cannot prove, since proof requires measurement along the capability axis, not at one point on it.

Appendix B — Pair-Free Translation (Stage B10): a boundary-mapping negative result

This appendix reports an additional experiment, run after the main study, testing the strongest form of the convergence claim: translation between the two embedding spaces without any paired examples (in the spirit of [vec2vec](#), 2025).

B.1 Design

The project's 4,000 image pairs were split into two disjoint image sets: MobileCLIP embeddings of images 0-1999 on one side, SigLIP embeddings of images 2000-3499 on the other - no image appears on both sides, so nothing links the sets except the shape of the spaces. A simplified, transparent translator was learned unsupervised: Gromov-Wasserstein optimal transport aligning the two intra-space distance matrices (the mathematical formalization of matching shapes without correspondences; [Alvarez-Melis and Jaakkola 2018](#)), followed by a CSLS-corrected mutual-nearest-neighbour self-learning loop refitting a ridge map. Grading used 500 true held-out pairs the training never saw, against a supervised ridge reference, a random-matrix floor, and chance (0.2%). Pre-registered criteria: PASS at $R@1 \geq 10\times$ chance and $\geq 25\%$ of supervised.

B.2 Result

Variant	Mean cosine	R@1	R@5	R@10
Supervised ridge (reference)	0.835	0.910	0.980	0.990
Unsupervised (no pairs ever)	0.395	0.002	0.004	0.006
Random matrix (floor)	0.017	0.000	0.010	0.014

Table B1: Pair-free translation graded on 500 held-out true pairs (chance $R@1 = 0.002$).

The outcome splits cleanly in two. **Distribution-level alignment was recovered from shape alone:** mean cosine to the true targets of 0.395 against a random floor of 0.017 - a 23x effect obtained without a single paired example. **Instance-level correspondence was not recovered:** $R@1$ remained at chance, and the self-learning loop converged to a stable, self-consistent but incorrect matching (428 mutual pseudo-pairs, frozen from iteration 4). The translator finds the right region of the target space but not the right points.

B.3 Bounded diagnostic study

Two hypotheses for the failure were tested and one survived. **(1) Calibration mismatch - refuted.** The conjecture that SigLIP's compressed cosine calibration (Section 4.2) breaks square-loss GW was tested on a synthetic testbed with simulated calibration compression: the method scored $R@1 = 0.92$ regardless, and the proposed normalization fix actively harmed performance. **(2) Manifold structure and anisotropy - supported.** The clustered synthetic testbed on which the method was validated ($R@1 = 144\times$ chance) was rebuilt to resemble real embeddings: a continuous manifold instead of discrete clusters, heavy-tailed density (hubness), and an anisotropy difference between the two spaces - the same property the Procrustes experiment measured between these very models (Section 4.3). On that testbed the method collapsed to chance, and a hyperparameter sweep (transport regularization, rank-normalized distances, larger samples) did not change the outcome. Discrete cluster structure gives shape-matching its anchor points; continuous, differently-stretched manifolds take them away.

B.4 Conclusion

Shape information alone suffices for coarse, distribution-level alignment between independently trained embedding spaces, but instance-level pair-free translation at this scale is beyond linear maps seeded by shape matching: the published vec2vec results employ adversarial training, cycle-consistency, vector-space-preservation losses and nonlinear adapters, and this experiment indicates that machinery is necessary rather than decorative. The finding composes with the main study: the Procrustes experiment showed the two spaces differ in anisotropy beyond rotation, and the diagnostic here shows that same property is plausibly what defeats cheap shape-only matching. This is the project's third boundary-mapping negative result, in the same pattern as the MLP comparison: the failed stronger method is itself the measurement - here, of exactly how much geometry alone can and cannot deliver.

Appendix C — Methodological corrections during the study

Two changes were made to Experiment A after its first run fell short of the pre-registered stitching criterion. Both are recorded here in full, with what changed, why, and what was deliberately left untouched, so that the final numbers can be read with the process that produced them visible.

C.1 What changed

Parameter	Run 1	Run 2 (final)	Reason
Sample count	1,646	10,000	1,646 passages give ~1,234 training rows for a 768x768 fit = 1.6 samples per input dimension. Ridge regression in that regime shrinks hard toward zero and reports systematically pessimistic held-out R2.
WikiText split	validation	train	Mechanical necessity, not a methodological choice: the validation split contains only ~1,646 passages over 100 characters, so 10,000 could not be drawn from it.
Everything else	-	unchanged	Same two models, same random-weights control, same pooling, same max length, same ridge penalty, same 75/25 fit/eval split, same metrics and thresholds.

Table C1: The complete difference between the two runs.

Which ratio governs, and why (clarifying Section 3.2). Section 3.2 states two numbers side by side - a 768x768 matrix of roughly 590,000 parameters, fitted on about 1,234 rows - and the relationship between them deserves to be explicit, because only one of them sets the statistical requirement. Multi-output ridge has the closed form $W = (X'X + \alpha I)^{-1} X'Y$, in which the inverted matrix depends on the inputs alone: the outputs never enter it. The problem therefore decomposes into 768 independent regressions, one per output dimension, each estimating 768 coefficients from the same design matrix and the same shared inverse. They do not interact and do not compete for data. The question that determines whether the fit is well-posed is asked once, per output regression: are there enough rows to estimate 768 coefficients? The governing ratio is therefore **rows per input dimension** - $1,234/768 = 1.6$ in run 1, and $7,500/768 = 9.8$ in run 2, against a conventional minimum of roughly 5 to 10. Rows divided by the 590,000 total parameter count (0.002 and 0.013) is not a difficulty measure: the fit is 768 tractable problems, not one intractable one. A useful consequence is that output width is nearly free, which is why the adapter of Experiment B - mapping 512 dimensions into 768 - needs rows relative to 512, and 3,000 image pairs sufficed there.

C.2 Why using the train split raises no leakage concern here

The train/validation/test convention exists to stop a *model being trained* from being evaluated on data it has already fitted. Each split has a role defined by how much information has leaked into the model: train (full leakage, by design), validation (indirect leakage, through the experimenter's tuning decisions), test (none, if used once). Those roles presuppose a training process. Experiment A has none.

In a normal training pipeline	In Experiment A
The model updates weights on the train split.	GPT-2 and Pythia are frozen. No gradient is ever computed. The text is probe stimuli - inputs used to elicit activations, like a stimulus set in a perception experiment.
The validation split guards hyperparameter choices.	No hyperparameters of the models are chosen; the only fitted objects are the stitching matrices in A3.
The test split gives the honest final number.	A3 holds out its own 25% of the 10,000 samples. Every reported R2 is measured on rows the ridge fit never saw. That is where held-out discipline lives in this study.

Table C2: The split names describe roles in WikiText's intended use (language-model benchmarking), not roles in this experiment.

Both splits are the same corpus, partitioned at the article level from the same set of verified Wikipedia articles and cleaned by the same pipeline; they differ only in which articles landed in which file. The distribution of the property this experiment actually depends on - well-formed English prose - is statistically indistinguishable between them. An analogy: to test whether two thermometers agree, one needs water samples at various temperatures; it is irrelevant which bucket the water came from, because those labels belonged to someone else's experiment.

C.3 A real asymmetry, disclosed

One genuine caveat exists in this area, and it is unrelated to the split choice. WikiText is Wikipedia text. Pythia was trained on The Pile, which *includes* an English Wikipedia component, whereas GPT-2's WebText was constructed with Wikipedia documents removed. Pythia has therefore likely encountered text of this kind during training and GPT-2 has not. Three points bound the consequence: the asymmetry applies identically to the validation and train splits, so it is not affected by the correction described here; the random-weights control processes exactly the same text, so the trained-minus-random gap remains a valid contrast; and the hypothesis under test concerns whether a linear map exists between two representations of the same input, not how surprising that input was to either model. The asymmetry is reported for completeness rather than because it is believed to bias the result.

C.4 Why this is diagnosis and not fitting the result

The distinction that matters: the change was motivated by a *diagnosed confound* identified before the second run, with an explicit prediction attached - if sample starvation was the cause, R2 would rise substantially; if the gap was real, it would plateau. Three pieces of evidence support the diagnosis rather than a lucky reroll. The arithmetic was checkable in advance (590K parameters, 1,234 rows). CKA, which is far less sample-hungry than regression, barely moved between runs (0.365 to 0.359 mean), indicating the underlying similarity was stable and only the regression estimate was distorted. And the control moved in the theoretically predicted direction (-0.773 to +0.173), since with adequate data ridge can exploit residual input correlations even through random layers. The run-1 numbers are reported in full in Section 3.2 rather than discarded.

C.5 One threshold reported as missed

For completeness: the pre-registered CKA criterion was written as 'diagonal CKA mean > 0.5 with random baseline < 0.15'. The final run produced a diagonal mean of 0.424 - below that line - with a random baseline of 0.067 and a trained/random ratio of 6.3x. The absolute threshold was set optimistically: absolute CKA between models with different tokenizers is expected to be modest, and the ratio to the control is the load-bearing test, since it is the quantity that isolates learning from architecture and input statistics. The overall verdict for Experiment A therefore rests on the ratio criteria and the stitching results, all of which passed decisively. This is recorded as a missed sub-threshold rather than a retro-fitted one.

C.6 Direction symmetry, and the anisotropy that masked it

The stitching fit in Section 3.3 maps GPT-2 into Pythia's space. Because the hypothesis is symmetric, the reverse fit should score comparably; a follow-up run tested every layer pair in both directions. Under the variance-weighted R2 used throughout, the reverse direction scored slightly higher in middle layers (mean 0.829 against 0.751) and lower at layer 0 (0.455 against 0.591), for a mean absolute per-layer difference of 0.054. Two further tests identified both effects, and neither reflects a failure of the isomorphism claim.

Layer 0 - a real information difference, architectural in origin. GPT-2 adds learned absolute positional embeddings at the embedding layer; Pythia applies rotary encoding inside attention, so its

layer 0 carries no positional term. Mean pooling therefore leaves a deterministic function of sequence length in GPT-2's pooled layer-0 vector and not in Pythia's. Measured directly: sequence length is recoverable from the pooled layer-0 vector with held-out $R^2 = 1.000$ for GPT-2 and 0.126 for Pythia. Predicting Pythia from GPT-2 is therefore unimpeded, while the reverse requires reconstructing information absent from its input - exactly the observed asymmetry, and the one place in this study where one space demonstrably contains something the other lacks.

Middle layers - an artifact of the scoring metric. Recomputing the same fits with uniform averaging across output dimensions instead of variance weighting collapses the gap: mean absolute difference falls from 0.097 to 0.006, and the two directions become indistinguishable (layer 4: 0.722 forward, 0.723 reverse; layer 5: 0.744 and 0.735; layer 6: 0.748 and 0.740). The cause is measurable anisotropy. Effective dimensionality (participation ratio of the covariance spectrum, out of 768) is far lower for GPT-2 than for Pythia at every depth:

Layer	GPT-2 effective dim	Pythia effective dim
0	8.9	92.7
4	3.3	9.3
8	10.9	19.8
12	2.2	43.9

Table C3: GPT-2's variance concentrates into as few as two or three effective directions - the documented rogue-dimension phenomenon, measured here in the project's own activations. A variance-weighted score with GPT-2 as target is therefore dominated by a handful of easily predicted directions, which inflates the reverse fit.

Conclusion. With the weighting artifact removed, the linear correspondence is symmetric in both directions, as the isomorphism hypothesis predicts, with a single architecturally explained exception at the embedding layer. The forward figures reported in Section 3.3 are essentially unchanged under either weighting (layer 4/5/6: 0.737/0.752/0.754 variance-weighted against 0.722/0.744/0.748 uniform), so the main result is robust to the metric choice. This check also connects the two experiments: the same property that inflates one direction here - unequal variance spread between two models' spaces - is what defeats rotation-only alignment in Experiment B.

Appendix D — A derived capability: cross-tier membership and similarity

This appendix extends the engineering discussion of Section 4.5 with a capability the shared index provides at no additional cost, identified after the main write-up.

Because the adapter places phone-side vectors in SigLIP's coordinates, the shared index supports **image-as-query** operations across tiers, not only text search. Computing $\hat{u} = L2(v \cdot W)$ for a photo on the device and querying the index with $\hat{u}$ itself yields, from the top of the ranking: **(a) membership and near-duplicate detection** — if the same photo, or a crop or re-save of it, is indexed on the server, its native SigLIP vector returns with a cosine far above the background, and thresholding on the gap (the triage rule of the B5 sandbox) answers 'is this already in the library?' without uploading the photo; and **(b) cross-tier similar-image search** — lower cosines with a clear ranking give 'server photos like this one', and symmetrically for server photos against phone-indexed content.

Two boundaries keep the claim precise. First, this predicts an image's *address* in SigLIP's space, never its content: W maps 512 numbers to 768 and cannot reconstruct pixels (though, per the security discussion in Appendix B, embeddings should still be treated as content-bearing, since dedicated inversion models recover approximate content from them). Second, the capability exists only through the adapter — a raw MobileCLIP vector compared against the server gallery scores exact chance (Section 4.3), so cross-tier membership is a property the matrix creates, not one the encoders share natively.

The expected accuracy is bounded below by the measured text-query results (Section 4.3), and image-to-image retrieval within one contrastive space is typically easier than text-to-image, since no modality gap is crossed. The measurement is a three-line variant of stage B3 — querying the SigLIP gallery with the adapted vectors instead of the caption embeddings — and is included in the B3 notebook as an optional cell.